
Software Requirements Specification

for

Dual Detect – Multi Modal Fake News Detection System Using AI

Version 1.0

Prepared by:

Shivam Sharma(2301010076)

Group Name: *<place your group name here>*

<name>	<student #>	<e-mail>
<name>	<student #>	<e-mail>
<name>	<student #>	<e-mail>
<name>	<student #>	<e-mail>
<name>	<student #>	<e-mail>

Instructor:	Ms. Radhika Gupta
Course:	B.Tech CSE Core
Lab Section:	<i><place your lab section here></i>
Teaching Assistant:	<i><place your TA's name here></i>
Date:	<i><place the date of submission here></i>

Contents

CONTENTS	II
REVISIONS	II
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE	1
1.2 PRODUCT SCOPE	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS	1
1.5 DOCUMENT CONVENTIONS	1
1.6 REFERENCES AND ACKNOWLEDGMENTS	2
2 OVERALL DESCRIPTION.....	3
2.1 PRODUCT OVERVIEW	3
2.2 PRODUCT FUNCTIONALITY	3
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	3
2.4 ASSUMPTIONS AND DEPENDENCIES.....	3
3 SPECIFIC REQUIREMENTS	4
3.1 EXTERNAL INTERFACE REQUIREMENTS	4
3.2 FUNCTIONAL REQUIREMENTS	4
3.3 USE CASE MODEL.....	4
4 OTHER NON-FUNCTIONAL REQUIREMENTS.....	6
4.1 PERFORMANCE REQUIREMENTS.....	6
4.2 SAFETY AND SECURITY REQUIREMENTS.....	6
4.3 SOFTWARE QUALITY ATTRIBUTES.....	6
5 OTHER REQUIREMENTS	7
APPENDIX A – DATA DICTIONARY	8
APPENDIX B - GROUP LOG	9

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Full Name	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Introduction

1.1 Document Purpose

This document specifies the functional and non-functional requirements for the Dual Detect application. It defines the system's objectives, architecture, performance expectations, and constraints, serving as a shared reference for project stakeholders, including developers, testers, and end-users. The SRS ensures that all parties have a clear, unambiguous understanding of what the system must achieve and the environment in which it will operate.

1.2 Product Scope

Dual Detect is a web-based, AI-driven platform for identifying fake news by analyzing both textual and visual content. The system integrates a Long Short-Term Memory (LSTM) model for text classification and a Convolutional Neural Network (CNN) for image analysis. A fusion mechanism combines the predictions to deliver a single authenticity score. Key capabilities include real-time detection, a simple and responsive user interface, and scalability to accommodate large datasets and multiple users. By merging state-of-the-art natural language processing with image recognition, Dual Detect aims to provide an effective tool for fact-checking organizations, media outlets, and the general public.

1.3 Intended Audience and Document Overview

- **Developers & Testers:** For implementation, quality assurance, and future maintenance.
- **End Users:** Journalists, fact-checkers, educators, and everyday users seeking quick credibility checks on news items.
This document describes the system's functionality, architecture, interfaces, and operational constraints, ensuring alignment among all stakeholders throughout the development lifecycle.

1.4 Definitions, Acronyms and Abbreviations

- **LLM:** Large Language Model
- **LSTM:** Long Short-Term Memory network
- **CNN:** Convolutional Neural Network
- **Flask:** Python web framework for backend APIs
- **API:** Application Programming Interface for client-server communication

1.5 Document Conventions

All requirements in this SRS are **numbered sequentially** and clearly categorized as either **Functional (FR)** or **Non-Functional (NFR)** to distinguish system capabilities from quality attributes.

- **Formatting:** Code snippets and command-line instructions are presented in *monospaced* font for easy readability and to avoid ambiguity.
- **Figures and Tables:** Every diagram or table is assigned a unique caption and cross-referenced in the text to maintain traceability.
- **Terminology:** Acronyms and key technical terms are introduced in **bold** on first use and remain consistent throughout the document.
- **Requirement Statements:** Each requirement is expressed in concise, testable language using “shall” or “must” to indicate mandatory behavior. Optional or future features are identified with “may” or “should.”

1.6 References and Acknowledgments

The development of **Dual Detect – Multimodal Fake News Detection System using AI** draws upon a variety of academic papers, software libraries, and official documentation. Principal sources include:

1. **TensorFlow & Keras Documentation** – Primary resources for implementing and tuning deep learning models, covering LSTM and CNN architectures used in this project.
2. **OpenCV Official Guide** – Detailed reference for image preprocessing tasks such as normalization, resizing, and augmentation.
3. **Research on Fake News Detection** – Key academic studies on natural language processing (NLP) and multimodal detection strategies, including transformer-based Large Language Models (LLMs) and comparative evaluations of classical ML techniques.
4. **Python Flask Framework Documentation** – Guidance on creating RESTful APIs and integrating machine learning models into a web application.
5. **Cloud Deployment Resources** – Heroku and AWS deployment manuals for scalable and secure hosting of AI applications.

2 Overall Description

2.1 Product Overview

The system receives text and associated images from users, preprocesses these inputs, and classifies the news as fake or real. A Flask-based web interface handles input and displays output, while backend LSTM and CNN models perform the core analysis. A weighted fusion mechanism merges the predictions, yielding a final authenticity score.

2.2 Product Functionality

The Dual Detect system provides a comprehensive set of functions to ensure accurate and efficient fake-news detection. It accepts textual content and associated images through an intuitive, browser-based form, then preprocesses the inputs—tokenizing and lemmatizing text while resizing and normalizing images. After preprocessing, the system generates predictions using two dedicated models: an LSTM network for textual analysis and a CNN for image classification. A weighted fusion algorithm combines these separate outputs into a single authenticity decision accompanied by a confidence score. All results are delivered in near real time, and the architecture supports multiple concurrent user requests without noticeable delay.

2.3 Design and Implementation Constraints

The application requires Python 3 and relies on TensorFlow/Keras libraries for model development and inference, with GPU resources recommended for efficient training and real-time performance. Success depends on the availability of high-quality, labeled datasets for both text and images. Deployment must be secure, protecting trained model files and APIs against unauthorized access or adversarial inputs.

2.4 Assumptions and Dependencies

It is assumed that users will access the system through modern web browsers with stable internet connections. The project also depends on continued availability of external datasets and fact-checking sources to support initial training and future model updates.

3 Specific Requirements

3.1 External Interface Requirements

The system provides two principal external interfaces.

- **User Interface (UI):** A responsive, browser-based web form allows users to enter a block of news text, upload an accompanying image, or provide both simultaneously. The UI must be intuitive, mobile-friendly, and accessible across major browsers.
- **Backend API:** A RESTful API built with Flask exposes endpoints to handle POST requests containing the user's text and image data. After processing, the API returns JSON-formatted predictions that include the fake/real classification, confidence values, and any relevant error messages.

3.2 Functional Requirements

The core functional capabilities of the system include the following:

1. **Input Handling:** Accept and validate user-submitted text and image files, ensuring that inputs meet defined format and size constraints.
2. **Text Preprocessing:** Perform tokenization, stopword removal, and lemmatization, then vectorize the text using suitable embedding techniques to prepare it for model inference.
3. **Image Preprocessing:** Resize and normalize uploaded images to standardized dimensions and color scales, applying data augmentation if needed to improve model robustness.
4. **Text Classification:** Analyze the preprocessed text using an LSTM-based model to produce a probability score for fake or real news.
5. **Image Classification:** Analyze the preprocessed image using a CNN to detect visual cues indicative of misinformation and generate a probability score.
6. **Result Presentation:** Display the final classification (fake/real) and confidence score on the UI almost instantly after input submission.

3.3 Use Case Model

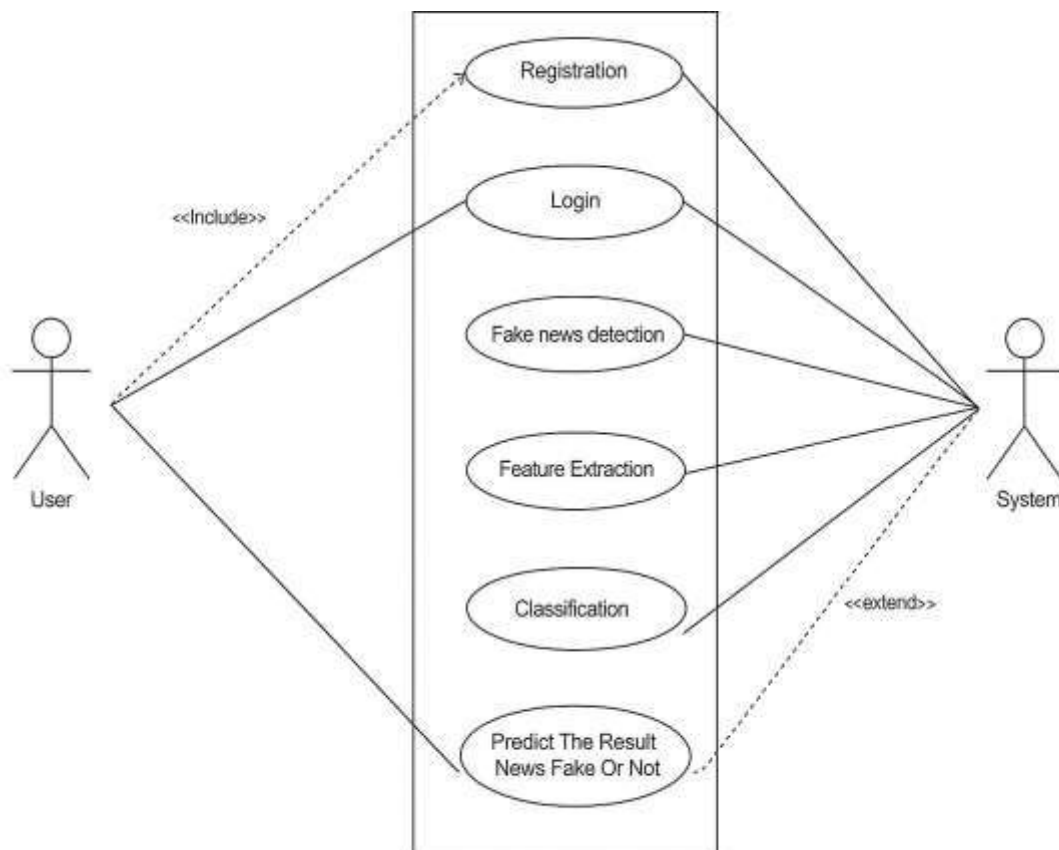
Use Case 1 – Submit News for Analysis

- **Actor:** End User (e.g., journalist, fact-checker, or general public).
- **Preconditions:** The application server is running, network connectivity is stable, and the web interface is accessible through a modern browser.

- **Main Flow:**

1. The user navigates to the Dual Detect web page and selects the option to analyze news.
2. The user enters a news headline or full article text and optionally uploads an associated image.
3. Upon submission, the system validates the input data and begins preprocessing: tokenizing and vectorizing the text while resizing and normalizing the image.
4. The preprocessed text is analyzed by the LSTM model, and the image is analyzed by the CNN model, each producing independent prediction scores.
5. A fusion component combines the two prediction scores into a single authenticity result with an overall confidence level.
6. The system sends the final classification back to the user interface.
7. The UI displays a clear message indicating whether the news is *fake* or *real*, accompanied by a confidence percentage and time of analysis.

Postconditions: The user receives an immediate, well-formatted result and may choose to submit additional analyses without refreshing or restarting the application.



4 Other Non-functional Requirements

4.1 Performance Requirements

The application is expected to deliver fast and consistent responses even under moderate load. Under normal network conditions, the system should provide a complete authenticity result within two seconds of receiving a request. It must also scale to accommodate at least 100 concurrent user sessions without significant performance degradation, ensuring that multiple journalists, fact-checkers, or casual users can access the service simultaneously without delays.

4.2 Safety and Security Requirements

All data exchanged between clients and the server must be transmitted over HTTPS to safeguard confidentiality and prevent unauthorized interception. To defend against common web vulnerabilities such as cross-site scripting or SQL injection, every user input will undergo strict validation and sanitization before processing. Furthermore, sensitive resources—including trained model files, API keys, and configuration settings—must be stored in secure, access-controlled environments to prevent tampering or theft. Regular security updates and vulnerability scans are required to maintain the integrity of the system over time.

4.3 Software Quality Attributes

The software must exhibit high reliability, consistently maintaining strong classification accuracy and fault tolerance even when handling unexpected inputs or system errors. Scalability is essential so that the application can seamlessly handle larger datasets and a growing number of users by leveraging cloud infrastructure and horizontal scaling techniques. A focus on usability ensures that the interface remains clear, intuitive, and responsive, allowing non-technical users to navigate easily and understand results without extensive guidance. Finally, the codebase should emphasize maintainability through a modular architecture, thorough inline documentation, and version-controlled repositories, enabling future developers to extend or enhance the system with minimal effort.

5 Other Requirements

Cloud Deployment:

- Host the application on a dependable cloud platform such as **Heroku, AWS, or Azure** for public access.
- Ensure high availability, automatic scaling, and support for continuous integration/continuous deployment (CI/CD) to enable seamless updates and minimal downtime.

Comprehensive Documentation:

- Provide **developer documentation** with setup instructions, environment configuration details, API references, and guidelines for training or retraining the AI models.
- Provide **user documentation** with step-by-step instructions for submitting text/images, interpreting authenticity results, and troubleshooting common issues.

Logging and Monitoring:

- Implement real-time **logging** to capture request details, system performance metrics, and error events while complying with data-privacy standards.
- Use monitoring tools (e.g., Prometheus, CloudWatch) to detect unusual activity, generate alerts for failures, and assist administrators in proactive maintenance.

Maintenance & Updates:

- Schedule regular updates for model retraining, security patches, and dependency upgrades.
- Maintain version control for code and documentation to support future enhancements and team collaboration.

Appendix A – Data Dictionary

- **text:** String – News content or headline submitted by the user for authenticity analysis. It supports up to approximately 10,000 characters and must be encoded in UTF-8.
- **image:** File – Optional image associated with the news content, typically in JPEG or PNG format. The file size should not exceed 5 MB and is automatically resized to 224 × 224 pixels for model processing.
- **label:** String – The final classification output generated by fusing the LSTM and CNN model predictions. Possible values are *fake* or *real*, and it is accompanied by a confidence score expressed as a percentage.
- **confidence_score:** Float – Indicates the probability (0.0–1.0) that the submitted news is fake or real. This value is displayed to the user as a percentage to convey prediction certainty.
- **timestamp:** DateTime – The system-generated time of analysis for each request, stored in ISO 8601 format to ensure consistency across logs and reports.
- **request_id:** String – A unique identifier automatically assigned to every submission, useful for tracking, auditing, and debugging individual analyses.
- **user_agent:** String – Captures information about the client’s browser or device, enabling analytics and troubleshooting without collecting personally identifiable information.
- **status:** String – Represents the processing state of each request, which can be *queued*, *processing*, *completed*, or *error*, helping administrators monitor the system’s operational flow.

Appendix B - Group Log

Week(s)	Activities & Deliverables
Weeks 1–2	Conducted requirement analysis, gathered multimodal datasets (text and images), and performed extensive data preprocessing including tokenization, lemmatization, and image normalization. Set up Git repository and initial project structure.
Weeks 3–4	Designed and trained the LSTM model for textual analysis and the CNN model for image classification. Tuned hyperparameters, evaluated performance, and created scripts for data augmentation to improve accuracy and robustness.
Week 5	Integrated trained models with the Flask web application . Developed RESTful APIs, built the responsive front-end interface, and performed unit and integration testing to ensure seamless end-to-end operation.
Week 6	Deployed the application on Heroku for public access, configured logging and monitoring tools, and created developer/user documentation. Performed final performance benchmarking and security audits.
Week 7 (Post-launch)	Monitored user feedback, fixed minor bugs, and scheduled periodic model retraining based on new data. Established a maintenance plan and future enhancement roadmap, including potential integration of transformer-based NLP models and advanced image feature extractors.